



PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG THÔNG TIN

Chương 5: Thiết kế kiến trúc và giao diện

ThS. Nguyễn Văn Hữu Hoàng

I NỘI DUNG

Chương này tập trung trình bày một số chủ đề sau đây:

- 5.1. Giới thiệu
- 5.2. Các bước trong pha thiết kế
- 5.3. Lựa chọn công nghệ
- 5.4. Thiết kế tương tranh và an toàn bảo mật
- 5.5. Phân rã hệ thống con
- 5.6. Thiết kế giao diện người dùng

| 5.1 GIỚI THIỆU

- **Phân tích** là để trả lời câu hỏi “**là gì/làm cái gì**” (“what”) – tập trung vào các yêu cầu (chức năng và phi chức năng) đối với hệ thống
- **Thiết kế** là để trả lời câu hỏi “**làm thế nào**” (“how”) – tập trung nghiên cứu sự thực thi của hệ thống.
 - Đưa ra những quyết định thiết kế phù hợp với các công nghệ được lựa chọn.
 - Đáp ứng các yêu cầu phi chức năng (vd: giao diện, hiệu năng, tính sẵn sàng...)
- Hoạt động chính của pha thiết kế là chuyển đổi các biểu diễn lớp phân tích thành các biểu diễn lớp thiết kế.
- Không có một quy tắc nào cho việc chuyển từ mô hình phân tích sang mô hình thiết kế. Vì nó là quá trình có tính sáng tạo liên quan đến kinh nghiệm của nhóm phát triển, công nghệ sử dụng cho phát triển, sở thích cá nhân...
- Vì phát triển phần mềm hướng đối tượng được tiến hành theo kiểu tăng dần, nên chúng ta không hy vọng có thể có một thiết kế đầy đủ cả hệ thống ngay từ đầu mà cần lên kế hoạch cho những phần của hệ thống theo thứ tự ca sử dụng ưu tiên (xanh, đỏ, vàng).
- Cần tìm một kiến trúc hệ thống có khả năng hỗ trợ một giải pháp hiệu quả, khả thi cho tất cả ca sử dụng. Sau đó, thiết kế chi tiết cho hầu hết các ca sử dụng quan trọng và thiết kế một phần cho các ca sử dụng ít quan trọng hơn. Điều chỉnh hợp lý trong những tiến trình lặp lại.

| 5.2 CÁC BƯỚC TRONG PHA THIẾT KẾ

- Pha thiết kế có thể được chia làm hai giai đoạn:
 - *Thiết kế kiến trúc hệ thống*
 1. Lựa chọn công nghệ mạng
 2. Thiết kế an toàn, bảo mật
 3. Phân rã hệ thống thành các hệ thống con
 4. Xây dựng biểu đồ gói
 - *Thiết kế chi tiết*
 - 1. Xây dựng biểu đồ lớp thiết kế
 - 2. Xây dựng biểu đồ tuần tự
 - 3. Xây dựng biểu đồ cơ sở dữ liệu

| 5.3 LỰA CHỌN CÔNG NGHỆ

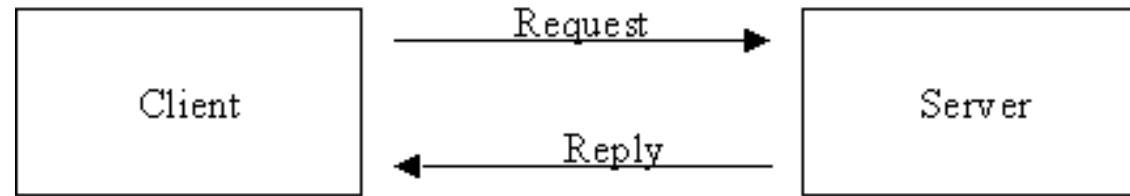
5.3.1 Lựa chọn công nghệ mạng

- Việc lựa chọn công nghệ mạng sẽ chỉ ra cách hệ thống được phân rã thành các thành phần logic và vật lý riêng biệt như thế nào. Kết quả của việc phân rã đó được gọi là hình trạng hệ thống (system topology).
- Để hiểu rõ hình trạng hệ thống, trước hết chúng ta sẽ khảo sát sơ lược lịch sử của các cấu trúc mạng và sau đó, xem xét các cấu trúc mạng hiện nay cũng như sự khác nhau giữa các kiểu máy khách client, giữa ứng dụng client- server với các ứng dụng phân tán khác. Cuối cùng sẽ xem xét biểu diễn hình trạng hệ thống bằng biểu đồ triển khai trong UML.

I Kiến trúc mạng đơn tầng và hai tầng

- **Kiến trúc một tầng** (one-tier Architecture) hay mô hình mainframe : Nghĩa là, bất cứ chương trình nào được đưa vào, chỉ có duy nhất một mức hoạt động tính toán chạy trên một máy. Kiến trúc một tầng không có mạng.
 - Ưu điểm chính là cài đặt đơn giản.
 - Nhược điểm chính của chúng là chỉ có thể tăng khả năng tính toán bằng cách mua thêm mainframe mới, hoặc nâng cấp cái cũ.
 - Ngày nay, kiến trúc mạng này vẫn còn được sử dụng cho các hệ thống nghiệp vụ cỡ lớn.
- **Kiến trúc hai tầng** (two-tier architecture) là thế hệ kiến trúc tiếp theo được phổ biến vào những năm 1970. Ý tưởng của kiến trúc này là nhằm tăng cường khả năng xử lý trên mỗi máy khách để cho các máy tính trung tâm không phải thực hiện toàn bộ các tiến trình.
 - Có thể thêm hay thay thế các máy khách rẻ hơn các máy tính trung tâm.
 - Các máy khách thường là các máy tính nhỏ hay các máy trạm, có thể truy nhập vào các máy tính trung tâm hay các server.
 - Vẫn còn được sử dụng rộng rãi hiện nay ví dụ như File Server.

| Hoạt động của mô hình kiến trúc hai tầng



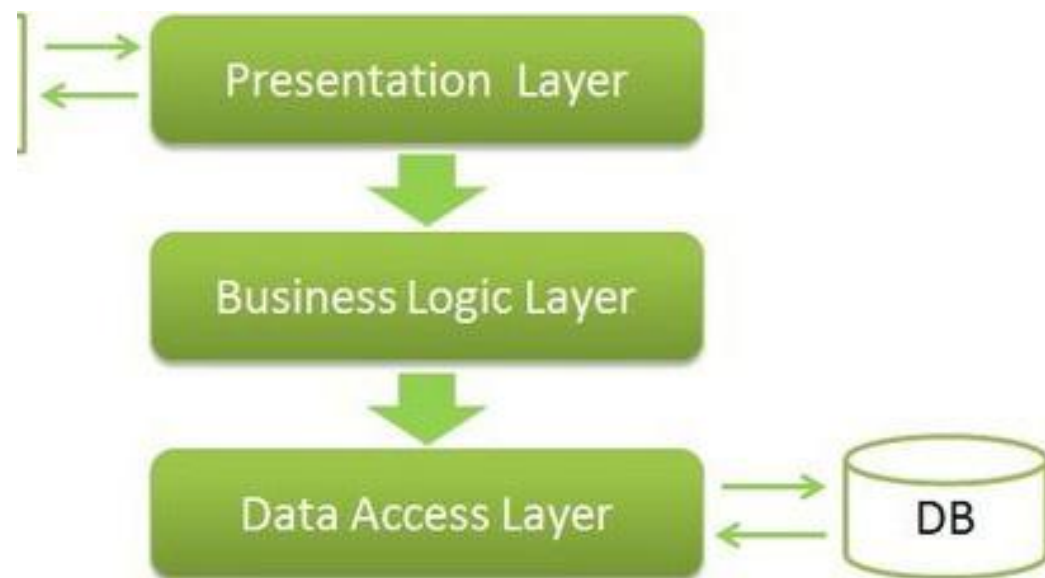
- *Máy khách* (Client) và *máy chủ* (Server) trao đổi thông tin với nhau dưới dạng các thông điệp (Message). Thông điệp gửi từ máy khách sang máy chủ gọi là các thông điệp yêu cầu (Request Message) mô tả công việc mà phần máy khách muốn máy chủ thực hiện.
- Mỗi khi máy chủ nhận được một thông điệp yêu cầu, máy chủ sẽ phân tích yêu cầu, thực thi công việc theo yêu cầu và gửi kết quả về máy khách (nếu có) trong một thông điệp trả lời (Reply Message).
- Mỗi một ứng dụng trên mô hình máy khách - máy chủ (client-server) phải có một Giao thức (Protocol) riêng để trao đổi thông tin, phối hợp công việc giữa máy khách và máy chủ. Thông tin trong giao thức gồm có:
 - Loại thông điệp, Định dạng thông điệp.
 - Số lượng và ý nghĩa của từng loại thông điệp.
 - Cách thức bắt tay, đồng bộ hóa tiến trình truyền nhận giữa máy chủ và máy Khách.

I Hoạt động của mô hình kiến trúc hai tầng

- Các chức năng về hoạt động nghiệp vụ có thể được cài đặt ở phần máy khách hoặc ở phần máy chủ và khi đó sẽ tạo ra hai loại kiến trúc:
 - *Máy khách nặng* (Fat Client): các hoạt động nghiệp vụ được cài đặt bên phía máy khách và phần máy chủ thực hiện chức năng chủ yếu về truy vấn và lưu trữ thông tin.
 - Ưu điểm: tạo ra ít giao tiếp trên mạng do dữ liệu tạm thời trong quá trình tính toán được lưu trên máy khách nên có thể giảm tắc nghẽn.
 - Nhược điểm là chi phí đầu tư phần cứng cho máy khách cao và khó nâng cấp.
 - *Máy Khách nhẹ* (Thin Client): tất cả máy tính để bàn của các nhân viên (khách) sẽ được kết nối vào một máy tính trung tâm (máy chủ). Máy chủ sẽ đóng vai trò như một kho tài nguyên phục vụ cho tất cả các máy khách nối với nó.
 - Ưu điểm: tiết kiệm chi phí, an toàn.
 - Nhược điểm: có khả năng dẫn đến tắc nghẽn, yêu cầu máy chủ phải có cấu hình cao.

| Kiến trúc ba tầng

- Kiến trúc ba tầng trở nên khá phổ biến vào những năm 1990 bằng cách chia hệ thống thành ba phần: giao diện người dùng, logic chương trình và lưu trữ dữ liệu.
- **Tầng giao diện người dùng:** thể hiện giao diện mà người sử dụng có thể nhập yêu cầu, dữ liệu và xem kết quả. Nó chỉ xử lý việc giao tiếp với người sử dụng, nhập xuất... mà không tính toán, kiểm tra, xử lý, hay các thao tác liên quan đến cơ sở dữ liệu.
- **Tầng ứng dụng:** tầng logic nghiệp vụ hay tầng dịch vụ để chạy chương trình đa luồng. Tầng này thực hiện xử lý các chức năng chính, kiểm tra các ràng buộc...Việc thực hiện này độc lập với cách thiết kế cũng như cài đặt giao diện và thông tin lấy từ tầng giao diện.
- **Tầng dữ liệu:** nhằm lưu trữ dữ liệu và cung cấp cơ chế an toàn cho việc truy nhập. Tầng này thực hiện các công việc liên quan đến dữ liệu mà phần mềm cần đến như đọc, ghi.

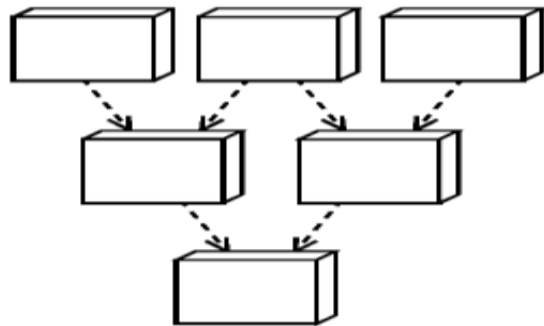


I Các ưu điểm của kiến trúc 3 tầng

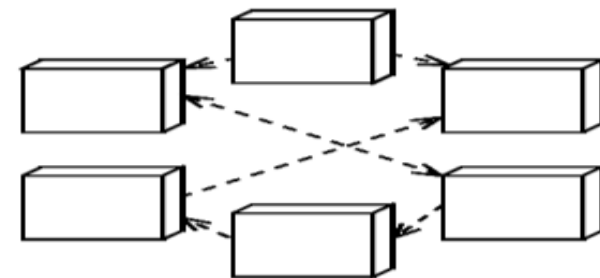
- **Dễ dàng khi phát triển:** Bất kỳ hệ thống nào cũng đều bao gồm ba phần: logic chương trình, giao diện người dùng và cơ chế quản lý dữ liệu. Việc phân chia hệ thống thành các phần như trên sẽ tạo điều kiện cho người lập trình thực hiện công việc một cách đơn giản hơn.
- **Sử dụng máy tính hiệu quả hơn:** Tùy theo từng tầng mà chúng ta sẽ sử dụng các máy tính sao cho phù hợp. Ví dụ, giao diện người dùng thực hiện nhiệm vụ đơn giản không đòi hỏi máy tính lớn; việc thực thi logic chương trình yêu cầu sử dụng CPU, bộ nhớ, ...
- **Cải tiến hiệu năng:** có thể nhân rộng các máy ở lớp dữ liệu và lớp giữa để lan truyền tính toán (cân bằng tải), mỗi tầng được chuyên môn hóa và như vậy sẽ dễ dàng tối ưu hóa.
- **Nâng cao tính bảo mật:** Có cơ chế bảo mật nghiêm ngặt để bảo vệ máy chủ, chương trình và dữ liệu.
- **Tính linh hoạt:** có thể thêm hoặc bớt các máy tính trong hệ thống nếu hệ thống đó được thiết kế theo kiến trúc ba tầng. Ví dụ, khi phần logic được thiết kế đúng đắn, chúng ta có thể phát triển nó theo kiến trúc một tầng, sau đó phát triển lên thành hai tầng, ba tầng tùy theo yêu cầu.
- **Đa dạng kiểu dáng máy client:** máy tính ở tầng client chỉ thực hiện nhận đầu vào và hiển thị kết quả trên màn hình. Do đó, chúng ta có thể sử dụng các thiết bị với các giao diện khác nhau như máy tính cá nhân, PDAs, mobile-phone.... Khi đó, tầng giữa và tầng dữ liệu vẫn làm việc như nhau, không có thay đổi.

| Kiến trúc phân tán

- Kiến trúc client – server có nghĩa là có một lượng lớn các client gửi yêu cầu đến một máy chủ lớn xử lý các yêu cầu đó.
- Ngược lại, kiến trúc phân tán (hay ngang hàng) được đặc trưng bằng một tập các máy tự chủ, truyền thông nhau theo bất kì một hướng nào khi có nhu cầu phát sinh.
- Kiến trúc phân tán thực hiện công việc tính toán lớn trải rộng ra trên nhiều máy tính trên mạng. Nghĩa là, khi có một lượng lớn dữ liệu cần thực hiện tính toán, chúng ta có thể chia dữ liệu hoặc tính toán đó thành nhiều bài toán tính toán trên nhiều máy độc lập.



Client – Server Architecture



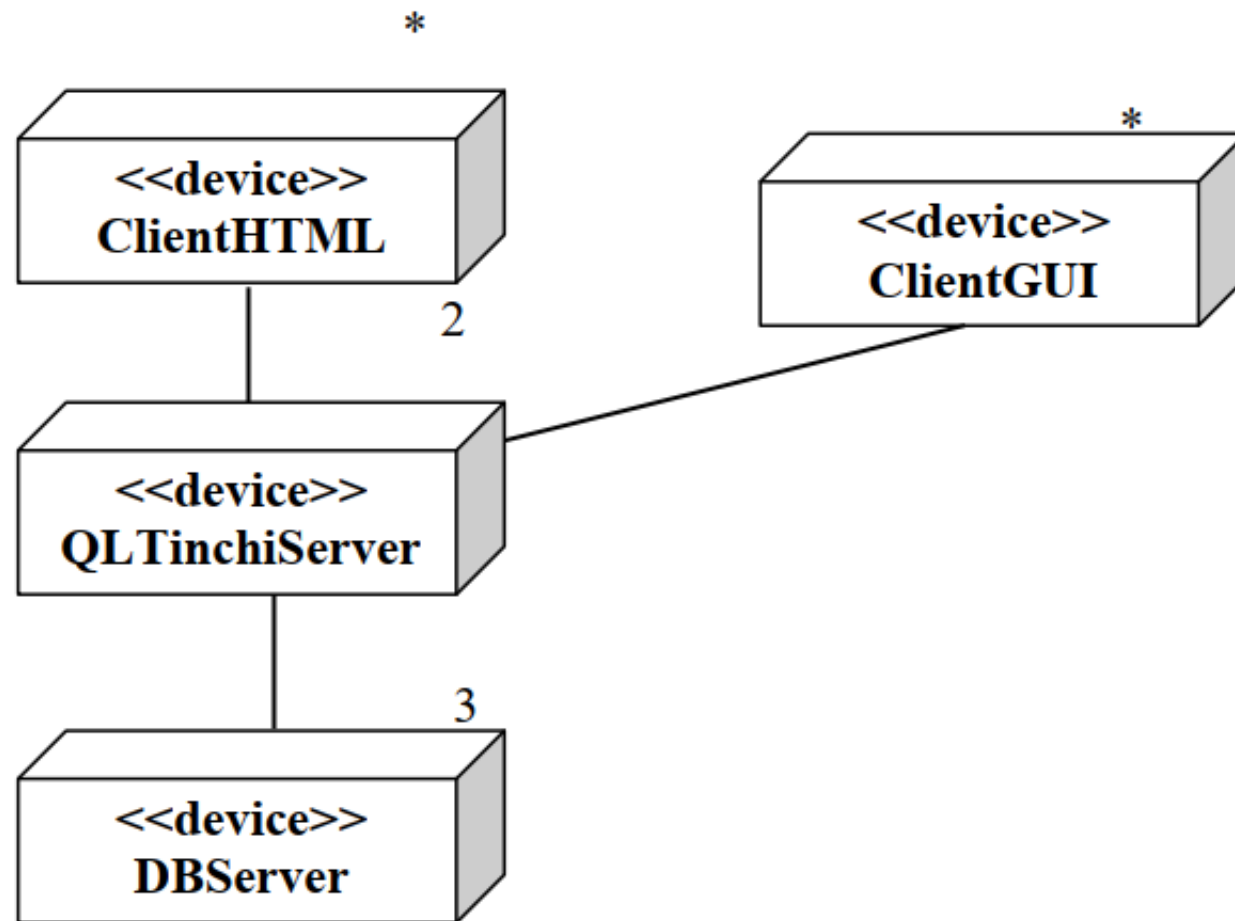
Distributed Architecture

| Biểu diễn hình trạng mạng với UML

- Biểu đồ triển khai đơn giản chỉ nêu ra các nút, đường truyền thông và vô số các điểm khác. Mỗi nút trong biểu đồ này thể hiện một máy trạm (thể hiện với từ khóa UML<<device>>).
- Một đường truyền thông chỉ ra rằng hai nút có thể liên lạc với nhau theo một cách nào đó. Các nút có dấu * thể hiện có rất nhiều nút có thể tồn tại trong thời gian chạy.
- Các biểu đồ triển khai giống như các biểu đồ lớp và biểu đồ đối tượng, trong đó chúng có thể nêu ra các kiến trúc có thể (kiểu nút) và các kiến trúc thực sự (các thể hiện nút). Khi chỉ ra các thể hiện nút, giống như các đối tượng trong biểu đồ đối tượng, nhãn nút có dạng tên:Kiểu và nên được gạch chân.

| Biểu đồ triển khai cơ bản cho Hệ quản lý học tín chỉ

- Tầng dữ liệu bao gồm 3 server cơ sở dữ liệu (gọi là DBServer).
- Tầng giữa, liên lạc với tầng dữ liệu bao gồm hai máy server (QLTinchiServer).
- Mỗi QLTinchiServer có thể được truy nhập đồng thời bởi bất cứ điểm ClientHTML hay GUIClient nào.

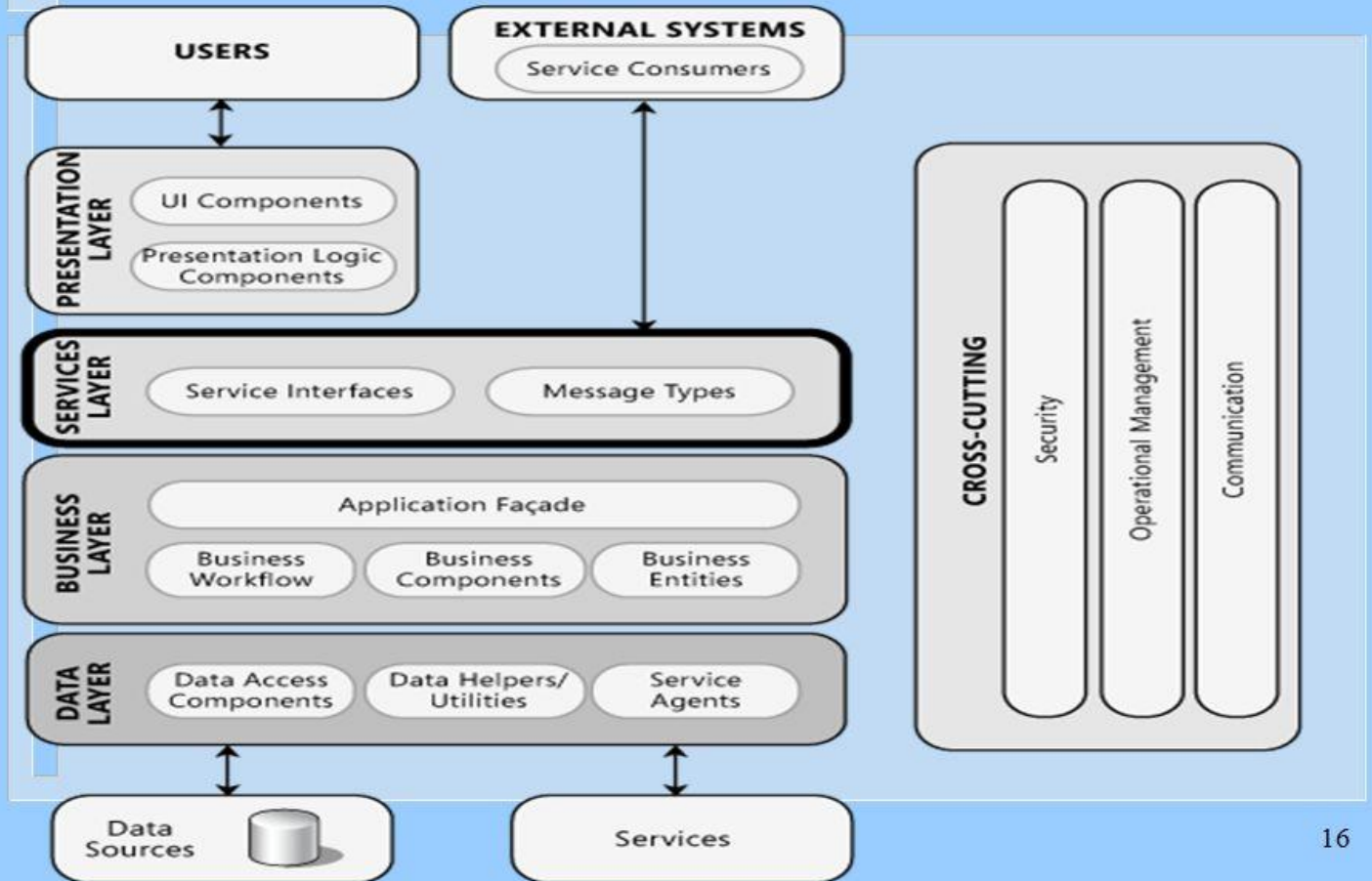


| 5.3.2 Kiến trúc phần mềm tổng quát

- **Tầng trình diễn** (presentation layer): Bao gồm các thành phần giao diện người dùng (UI: User Interface) và các thành phần logic trình diễn (Presentation logic).
- **Tầng dịch vụ** (service layer): Bao gồm các giao diện dịch vụ và các kiểu dịch vụ.
- **Tầng nghiệp vụ** (business layer): Bao gồm các thành phần, thực thể nghiệp vụ và luồng hoạt động nghiệp vụ.
- **Tầng dữ liệu** (data layer): Bao gồm các thành phần truy nhập dữ liệu, các tiện ích dữ liệu và các tác nhân dịch vụ.
- **Tầng kết nối** (cross cutting): Bao gồm an toàn, truyền thông và quản lý điều hành.

KIẾN TRÚC PHẦN MỀM TỔNG QUÁT

Layered Software Architectural styles Example of Web Applications Architecture Style

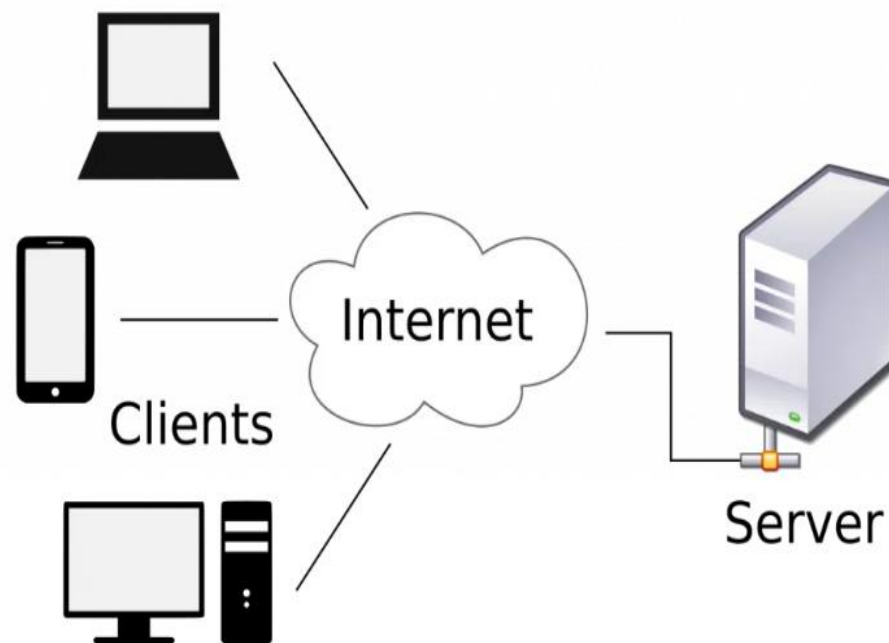


| Các kiểu kiến trúc phần mềm

Phân loại	Kiểu kiến trúc
Giao tiếp	Kiến trúc hướng dịch vụ, Kiến trúc bus thông điệp
Triển khai	Client/Server, N-tầng, 3-Tầng
Miền	Thiết kế hướng miền
Cấu trúc	Dựa vào thành phần, Hướng đối tượng, Kiến trúc theo tầng

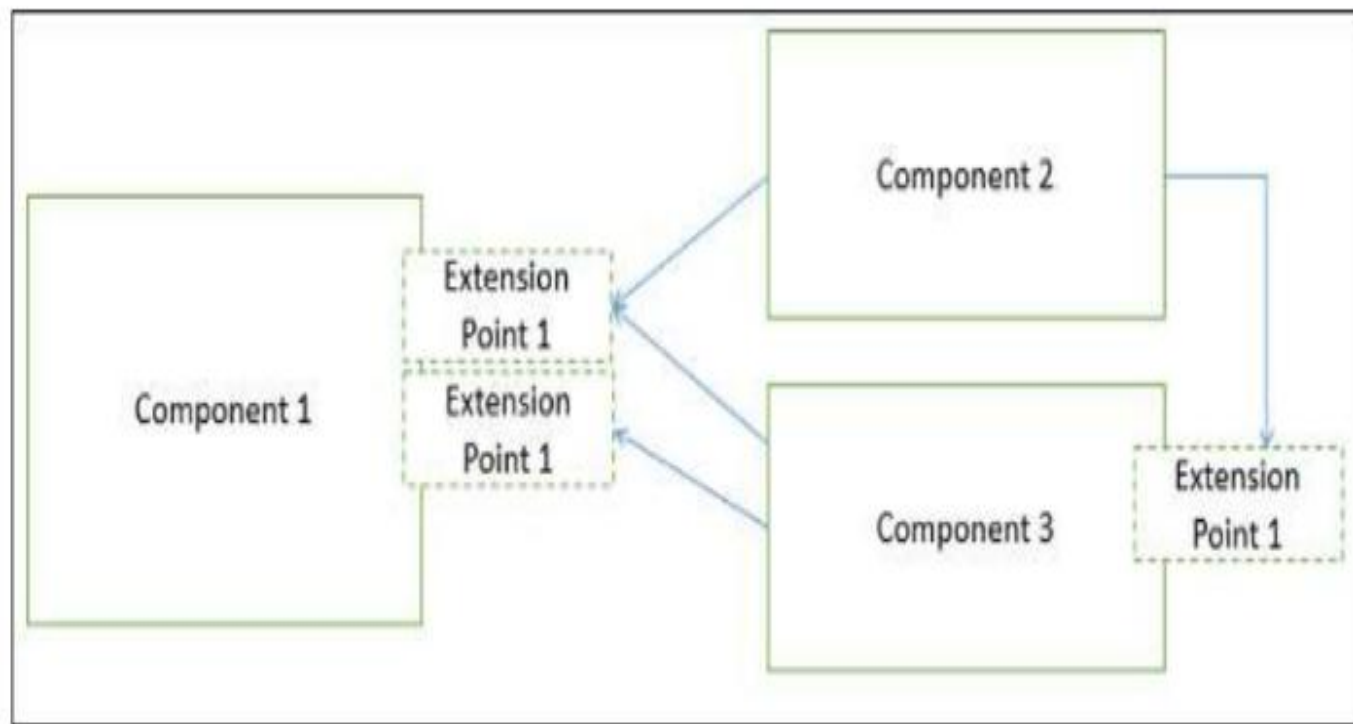
| Kiểu kiến trúc Client/Server

- Kiến trúc client/server mô tả hệ thống phân tán bao gồm các hệ thống máy chủ, các máy khách và các mạng lưới kết nối.
- Ưu điểm:
 - Có tính bảo mật cao.
 - Truy cập dữ liệu tập trung.
 - Dễ dàng bảo trì.
- Nhược điểm:
 - Dễ tắc nghẽn lưu lượng.
 - Độ bền phụ thuộc vào server.
 - Chi phí bảo trì khá cao.



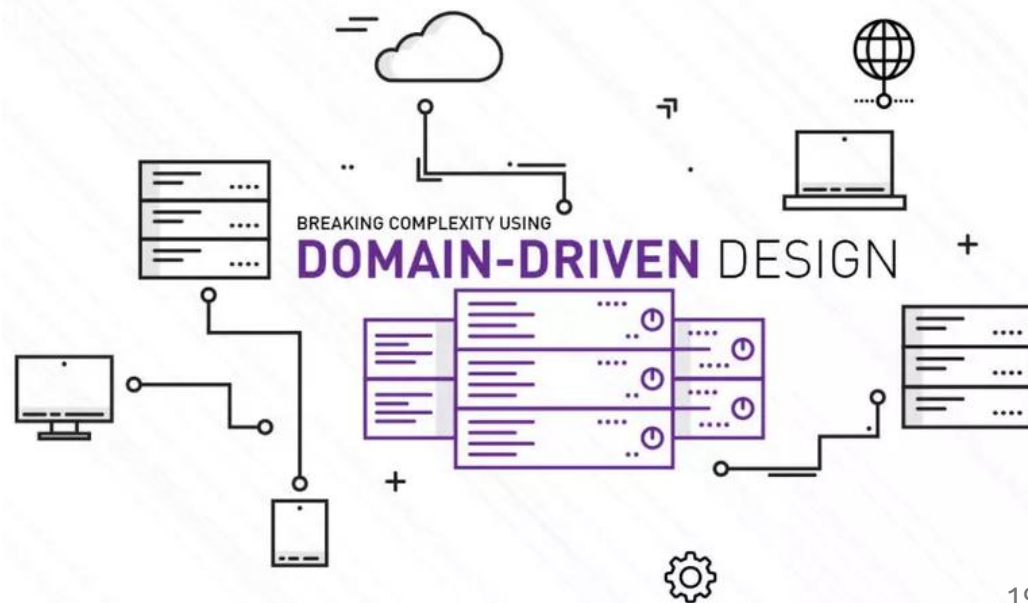
| Kiểu kiến trúc dựa vào thành phần

- Phân rã kiến trúc thành các thành phần chức năng riêng biệt hoặc các thành phần tiếp xúc với giao diện chuẩn qua kết nối.
- Nguyên tắc của thiết kế thành phần:
 - Khả năng tái sử dụng.
 - Khả năng thay thế được.
 - Khả năng mở rộng.
 - Được đóng gói.
 - Tính độc lập.
- Ưu điểm:
 - Dễ triển khai.
 - Chi phí thấp hơn.
 - Dễ phát triển.
 - Tái sử dụng.
 - Giảm sự phức tạp kỹ thuật.



| Kiểu kiến trúc hướng miền

- Là một cách tiếp cận hướng đối tượng để thiết kế phần mềm dựa trên các hoạt động nghiệp vụ với các thuộc tính, hành vi và quan hệ giữa chúng.
- Để áp dụng thiết kế dựa trên mô hình miền, chúng ta phải có một hiểu biết khá rõ về lĩnh vực nghiệp vụ mà mình muốn mô hình, hoặc có kỹ năng cũng như kiến thức nghiệp vụ liên quan.
- Ưu điểm:
 - Truyền thông liên lạc tốt.
 - Dễ dàng mở rộng.
 - Dễ dàng kiểm chứng các thành phần.

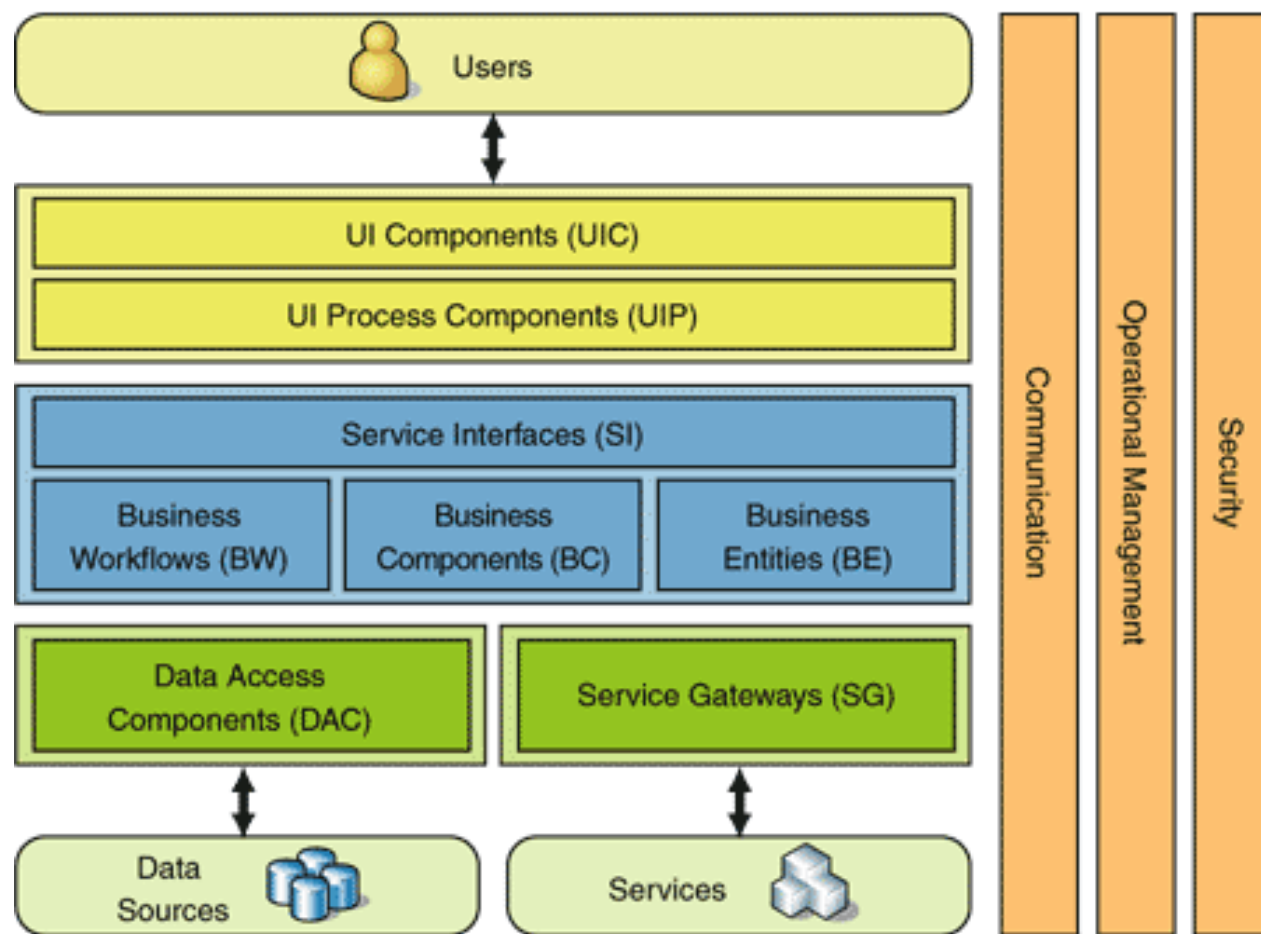


| Kiểu kiến trúc phân tầng

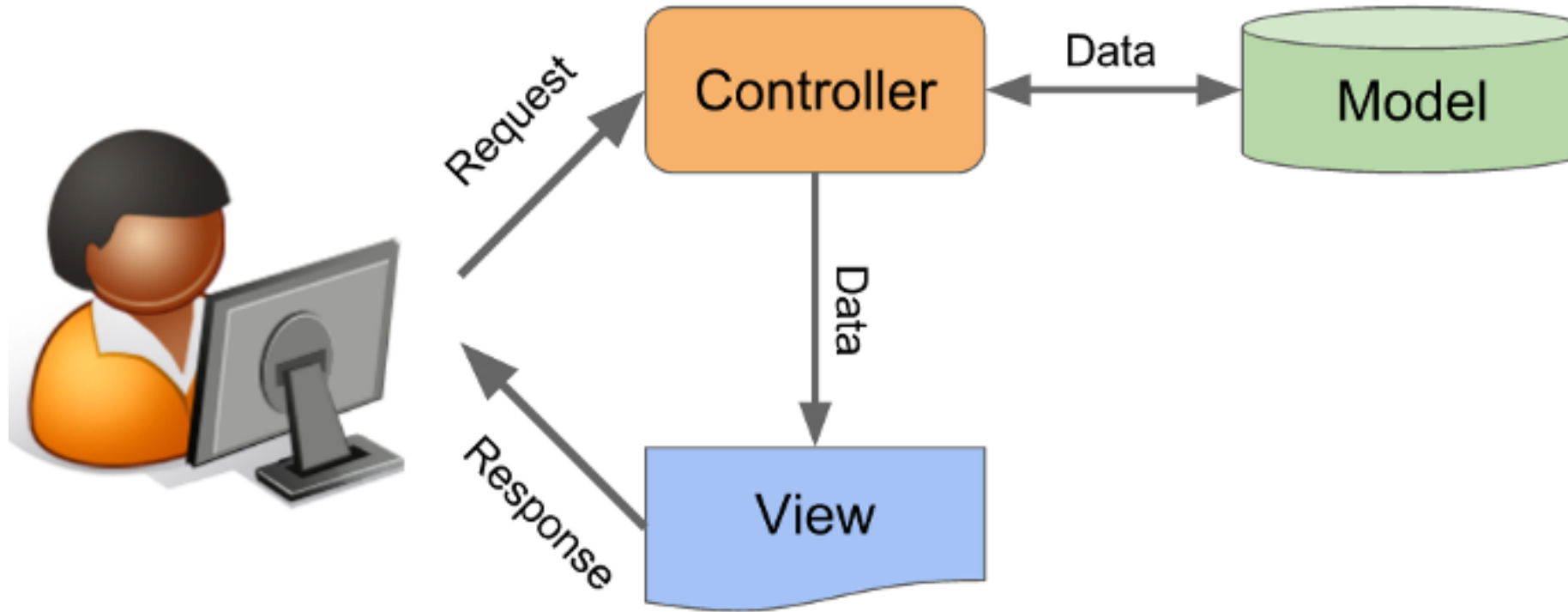
- Các nhóm chức năng có liên quan trong một ứng dụng thành các tầng riêng biệt được xếp chồng lên nhau theo chiều dọc.
- Các tầng của một ứng dụng có thể nằm trên các máy tính vật lý như nhau hoặc có thể được phân phối trên các máy tính riêng biệt.
- Nguyên tắc thiết kế phân tầng:
 - Trừu tượng.
 - Đóng gói.
 - Xác định rõ ràng các tầng chức năng.
 - Kết dính cao.
 - Tái sử dụng.
 - Kết nối lỏng lẻo.
- Ví dụ mô hình MVC là kiểu kiến trúc phân tầng.

| Kiểu kiến trúc phân tầng

- Ưu điểm:
 - Trừu tượng.
 - Cô lập các tầng.
 - Dễ dàng quản lý.
 - Cải thiện hiệu suất.
 - Dễ kiểm thử.



I VÍ DỤ MÔ HÌNH MVC



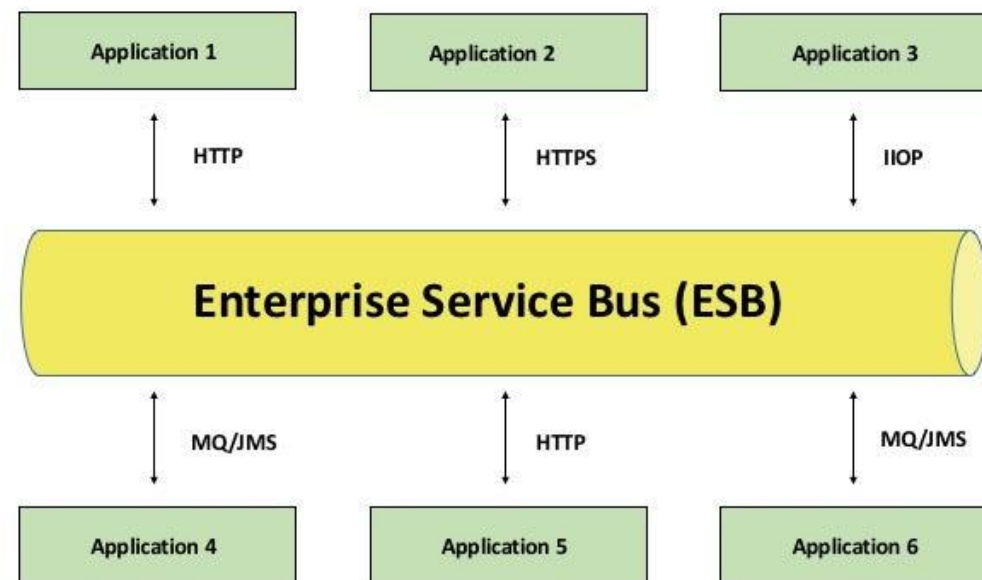
Model (dữ liệu): Quản lý xử lý các dữ liệu.

View (giao diện): Nơi hiển thị dữ liệu cho người dùng.

Controller (bộ điều khiển): Điều khiển sự tương tác của hai thành phần Model và View.

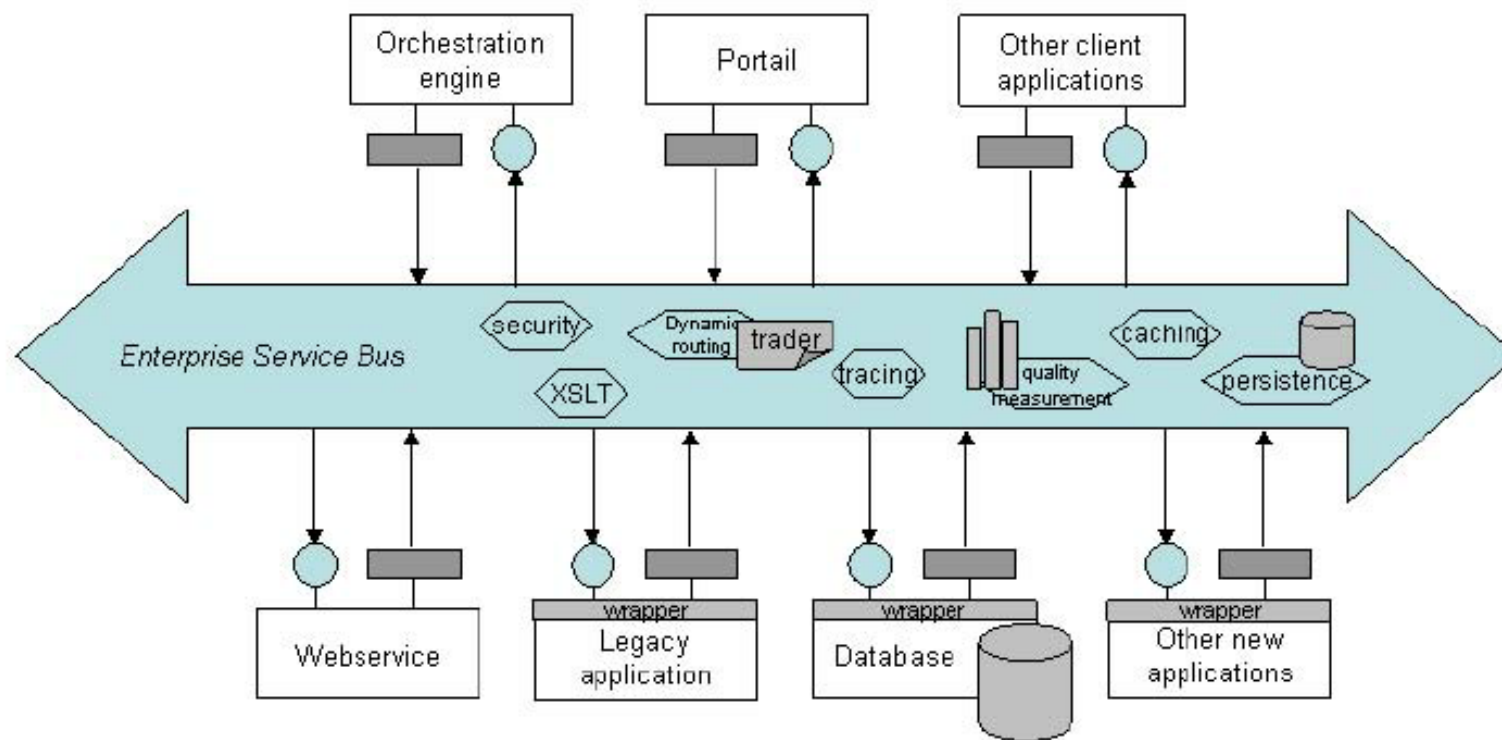
| Kiểu kiến trúc bus thông điệp

- Kiến trúc bus thông điệp mô tả các nguyên tắc sử dụng một hệ thống phần mềm có thể nhận và gửi thông điệp qua một hoặc nhiều kênh truyền thông
- Một bus thông thường cung cấp khả năng xử lý :
 - Giao tiếp hướng thông điệp.
 - Xử lý logic phức tạp.
 - Chỉnh sửa cho xử lý logic.
 - Kết hợp nhiều môi trường khác nhau (Java, .NET ...).



| Kiểu kiến trúc bus thông điệp

- Ưu điểm:
 - Khả năng mở rộng cao.
 - Ít phức tạp.
 - Linh hoạt.
 - Kết nối lỏng.
 - Ứng dụng đơn giản.



| Kiểu kiến trúc hướng dịch vụ (SOA)

- Kiến trúc hướng dịch vụ (Service Oriented Architecture) cho phép ứng dụng cung cấp các chức năng như một tập hợp các dịch vụ và tạo ra các ứng dụng sử dụng dịch vụ.
- Nguyên tắc của kiến trúc SOA:
 - Dịch vụ là tự trị.
 - Dịch vụ liên kết lỏng lẻo.
 - Khả năng tương thích dựa trên chính sách.
- Ưu điểm:
 - Rút ngắn thời gian đưa ra thị trường.
 - Bảo trì hiệu quả.
 - Khả năng thích ứng cao hơn.



| 5.4 THIẾT KẾ TƯƠNG TRANH VÀ AN TOÀN-BẢO MẬT

5.4.1 Thiết kế tương tranh

- **Hệ thống tương tranh** (concurrent system) là hệ thống trong đó có nhiều hoạt động thường xảy ra cùng một lúc.
- Tính toán tương tranh xem xét một số vấn đề:
 - Làm sao đảm bảo được rằng thông tin được cập nhật đầy đủ trước khi có người thao tác trên cập nhật đó? Ví dụ, dừng mọi truy nhập vào đăng ký môn học cho tới khi tất cả các chi tiết môn học đã được thêm vào.
 - Làm sao đảm bảo được rằng thông tin không được cập nhật trong khi nó đang được đọc? Ví dụ, không cập nhật môn học trong khi nó đang được bạn đọc xem.
- Cách dễ dàng nhất để xử lý tương tranh là thêm ràng buộc vào hệ thống hay đưa thêm vào quy tắc nghiệp vụ.
- Hoặc cũng có thể áp dụng kỹ thuật “Transaction” để xử lý tương tranh trong ngôn ngữ lập trình.

I Ví dụ

- **Vấn đề:** Xem xét một hệ thống mua bán vé máy bay. Khách hàng A tới một văn phòng đặt vé cùng thời điểm với khách hàng B tới một văn phòng đặt vé khác. Cả hai cũng định mua một vé cho cùng chuyến bay đi Đà Nẵng. Thật không may, **chỉ còn lại một vé duy nhất**. Cả hai cùng lúc hỏi “Có còn vé không?” Cả hai nhân viên bán vé sẽ cùng kiểm tra trên hệ thống và sẽ cùng trả lời “Còn vé”.
- ➔ Khi có sự tranh chấp như vậy thì chúng ta phải đảm bảo, ít nhất là không tình cờ bán hai vé khi chỉ còn một ghế duy nhất.
- **Giải pháp:** Để tránh điều này, chúng ta có thể đưa ra thêm quy tắc nghiệp vụ như sau: khi một nhân viên truy vấn và giữ chỗ vé còn lại, nó tạm thời được giữ cho tới khi nhân viên hủy bỏ yêu cầu hoặc thời hạn đặt vé đã hết (ví dụ, việc đặt vé tạm thời được phép kéo dài 10 phút nếu nhân viên không hủy bỏ nó trước).

| 5.4.2 Thiết kế an toàn – bảo mật

- Một hệ thống được gọi là an toàn khi hệ thống được bảo vệ khỏi sự lạm dụng dù là vô tình hay cố ý. An toàn là thuật ngữ rộng hơn rất nhiều, nó có thể được chia thành năm khía cạnh:
 - **Sự riêng tư** (Privacy): Thông tin có thể được che dấu, và chỉ ở trạng thái sẵn sàng với những người dùng được phép tác động vào nó như xem hoặc chỉnh sửa. Điều này đảm bảo cho người sử dụng khai thác tài nguyên của hệ thống theo đúng chức năng, nhiệm vụ đã được phân cấp, ngăn chặn được sự truy nhập thông tin bất hợp pháp.
 - **Xác thực** (Authentication): Cần biết nơi mà phần thông tin được gửi đến để quyết định xem phần thông tin đó có đáng tin cậy hay không.
 - **Tính không thể bác bỏ được** (Irrefutability): Ngược với xác thực, tính không thể bác bỏ được nhằm đảm bảo rằng người tạo ra thông tin không thể phủ nhận rằng họ chính là người tạo ra nó. Điều này hữu ích cho chúng ta khi có bất kì sai sót xảy ra.
 - **Tính toàn vẹn** (Integrity): Đảm bảo rằng thông tin không bị mất mát trên đường đi, bảo đảm sự nhất quán của dữ liệu trong hệ thống. Có thể đưa ra các biện pháp để ngăn chặn được việc thay đổi bất hợp pháp hoặc phá hoại dữ liệu.
 - **Tính an toàn** (Safety): có thể điều khiển việc truy cập tài nguyên (như máy móc, tiến trình, cơ sở dữ liệu và các tệp). Tính an toàn cũng được hiểu như là quyền hạn.

| Những quy luật chung về an toàn-bảo mật

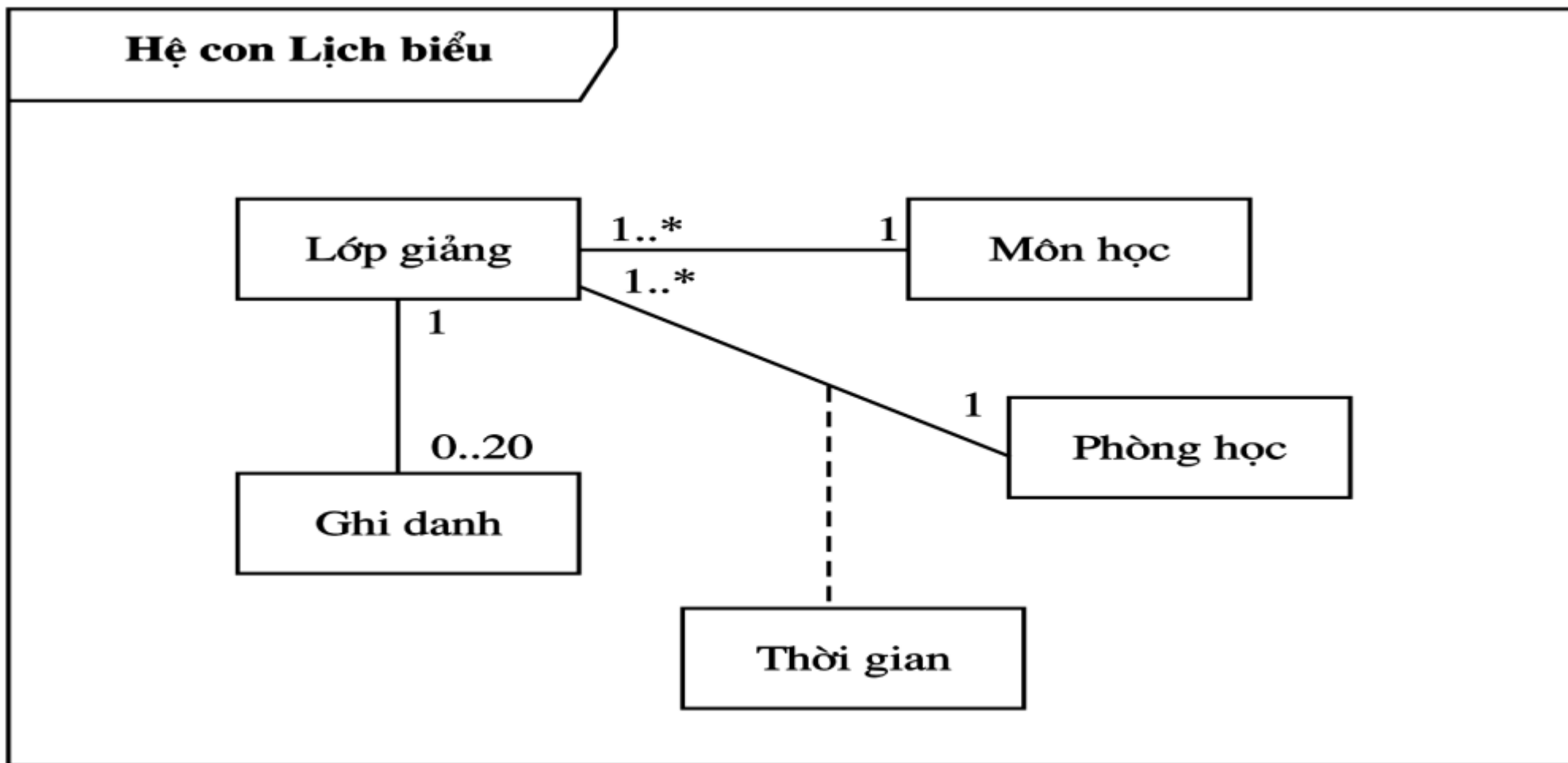
- Ngăn chặn việc xâm nhập máy chủ khi không được phép.
- Các thông tin nhạy cảm như chi tiết về ý tưởng kinh doanh hay chiến lược kinh doanh, các hồ sơ cá nhân, chi tiết về số thẻ tín dụng, các thông tin liên quan đến an ninh quốc gia.... phải được đảm bảo không bị truyền ra ngoài.
- Đảm bảo thông tin khi đi ra bên ngoài không bị “nghe lén” trong quá trình truyền và chỉ có đúng người nhận mới có thể đọc được.
- Bảo vệ mật mã (password) của khách hàng và nhân viên, nó không chỉ đơn thuần là chính sách bảo mật mà nó còn liên quan tính riêng tư cao.
- Bảo vệ tài nguyên hệ thống của client: bảo vệ client chống lại các truy cập tài nguyên bất hợp pháp và chống lại sự phá hoại từ bên ngoài.

| 5.5 PHÂN RÃ HỆ THỐNG THÀNH CÁC HỆ THỐNG CON

5.5.1 Phân rã thành các hệ thống con

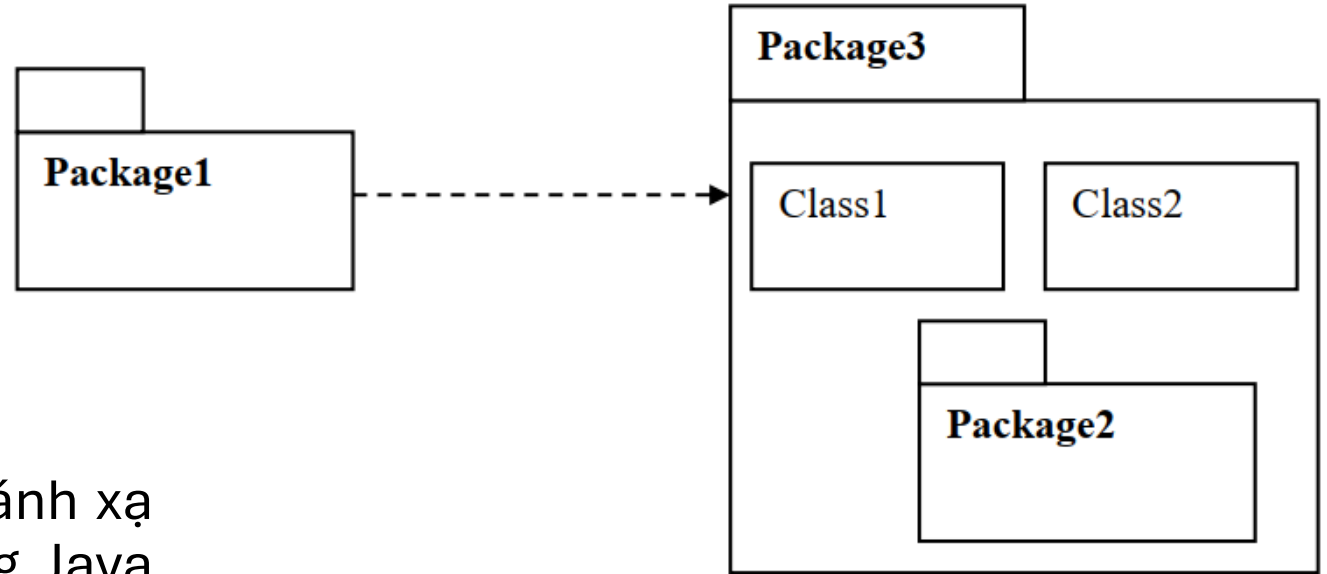
- Hệ thống con (subsystem) là sự gom nhóm một cách logic các lớp có sự gắn kết mạnh bên trong (của hệ thống con) và sự liên kết yếu bên ngoài (giữa các hệ thống con).
- Khái niệm gói (package) trong UML cho phép chúng ta gom các lớp có liên quan với nhau. Biểu đồ gói có thể chứa các lớp hay các gói khác
- Ta cũng có thể biểu diễn hệ thống con dưới dạng gói, mang theo khuôn mẫu `<<subsystem>>`.
- Nội dung của một hệ thống con (gồm các lớp và các mối liên quan giữa chúng) được UML 2.0 diễn tả trong một khung (frame), với một tựa đề viết trong một hình chữ nhật cắt góc theo khuôn dạng: `[<loại>] Tên [<tham số>]`
- Số các lớp trong một hệ thống con không nên ít quá hay nhiều quá.

| Hệ thống con



| Sơ đồ gói

- Một gói có thể sử dụng để biểu diễn:
 - Một cụm (layer)
 - Một hệ thống con (subsystem)
 - Thư viện dùng lại (resusable library)
 - Khung (framework)
 - Các lớp cần triển khai cùng nhau.
- Từ quan điểm lập trình, các gói có thể ánh xạ thành các cấu trúc như package trong Java hoặc namespace trong C#.
- Gói là khái niệm trong thời gian biên dịch và vì vậy nó tiện lợi trong tổ chức mã cho phát triển, triển khai và bảo trì.



| Phân rã thành các hệ thống con

Sự gắn kết mạnh của các lớp trong cùng một hệ thống con thể hiện:

- **Về mục đích:** Các lớp phải cung cấp các dịch vụ có cùng bản chất cho người dùng. Như vậy chúng phải thuộc vào cùng một lĩnh vực và đề cập một số thuật ngữ chung (chẳng hạn hệ thống con giao diện đề cập các thuật ngữ như: cửa sổ, thực đơn, nút nhấn,...)
- **Về xu thế phát triển:** Người ta tách các lớp bền vững với các lớp có nhiều khả năng thay đổi. Đặc biệt, thường tách các lớp nghiệp vụ với các lớp ứng dụng, và xếp chúng vào các hệ con khác nhau.
- **Về ứng dụng các công nghệ:** Để tận dụng các công nghệ có sẵn, như các thư viện chương trình (lớp/thành phần), các GUI, các hệ quản trị cơ sở dữ liệu,..., ta thường tách các hệ thống con giao tiếp, hệ thống con quản trị dữ liệu ra khỏi phần lõi (ứng dụng và nghiệp vụ) của hệ thống.
- Sự liên kết giữa các hệ thống con thể hiện ở mối liên quan phụ thuộc giữa chúng.

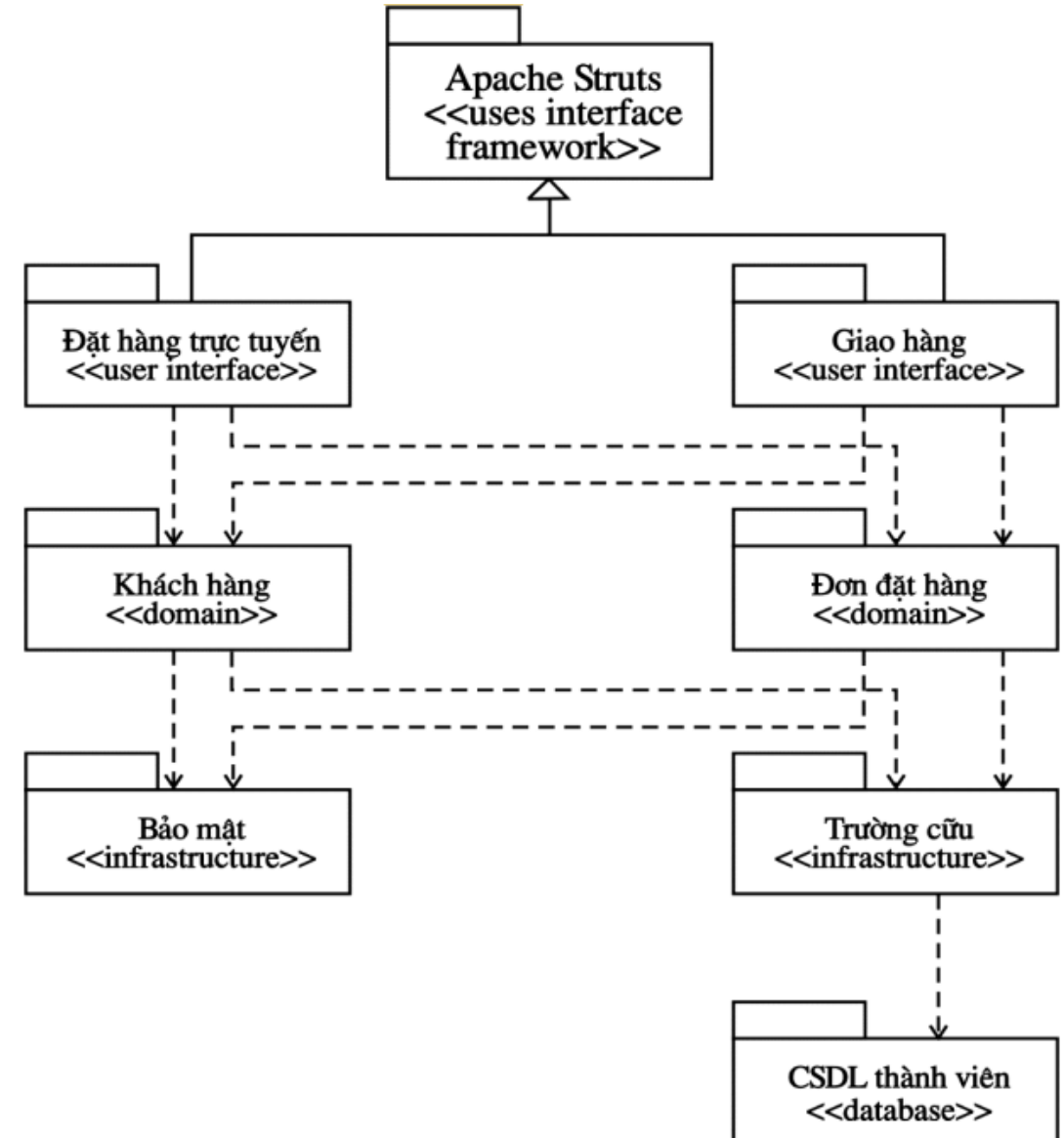
| Phân rã thành các hệ thống con

Có hai cách để chia hệ thống thành các hệ thống con:

- Tổ chức hệ thống thành các **tầng**, với mối quan hệ khách/chủ luôn luôn hướng từ tầng trên xuống (các) tầng dưới.
 - Tầng trên đóng vai trò khách, tầng dưới đóng vai trò chủ.
 - Ví dụ: Hệ thống tạo cửa sổ trong giao diện người dùng của máy tính.
- Tổ chức hệ thống thành các **lát**, với mối quan hệ ngang hàng giữa các lát; tuy nhiên các **lát** là khá độc lập hoặc liên kết yếu với nhau.
 - Ví dụ: Hệ điều hành thường gồm các hệ con như là các hệ quản lý tệp, hệ điều khiển thiết bị, hệ quản lý sự kiện ..
- Tổ chức phân tầng là đáng được ưu tiên hơn, vì nó mang lại nhiều ưu thế trong thiết kế, trong cài đặt, cũng như trong việc sử dụng lại.
- Đối với các hệ thống lớn, thì ta thường phải phối hợp cả 2 cách tổ chức phân tầng và phân lát.
 - Phân hệ thống thành các tầng (architectural building layers)
 - Trong mỗi tầng, thì lại phân thành các lát (architectural building blocks)

| Phân rã thành các hệ thống con

- Trong các hệ thống lớn, số tầng còn có thể nhiều hơn 3.
- Minh họa về kiến trúc khách/chủ gồm 5 tầng.
- Trong đó mỗi gói (hệ thống con) đều có mang khuôn mẫu thích hợp:
 - <<user interface framework>>: khung giao diện người dùng
 - <<user interface>>: giao diện người dùng
 - <<domain>>: lĩnh vực
 - <<infrastructure>>: cơ sở hạ tầng
 - <<database>>: cơ sở dữ liệu



| 5.5.2 Mô tả các thành phần vật lý của hệ thống

Thành phần (Components) và biểu đồ thành phần (Component diagram):

- Nếu như biểu đồ gói (hệ thống con) mà ta nói ở phần trên phản ánh cho góc nhìn về cấu trúc logic của hệ thống (ở mức cao so với biểu đồ lớp), thì biểu đồ thành phần cho ta một cách nhìn về cấu trúc vật lý của hệ thống.
- Chữ "vật lý" ở đây được hiểu theo nghĩa là sự mô tả hướng tới các sản phẩm phần mềm, là kết quả của sự cài đặt và thực sự tồn tại, chứ không phải là các sản phẩm logic.
- Tuy nhiên, ở đây ta chưa đề cập tới phần cứng, mặc dù tính vật lý của nó cũng là đương nhiên.
- UML định nghĩa **thành phần** (component) là một bộ phận vật lý và thay thế được của hệ thống, cung cấp sự thực hiện (implementation) cho một tập các giao diện (interfaces)
- Nói cách khác, thì thành phần là một cài đặt của một tập hợp các phần tử logic.

| Mô tả các thành phần vật lý của hệ thống

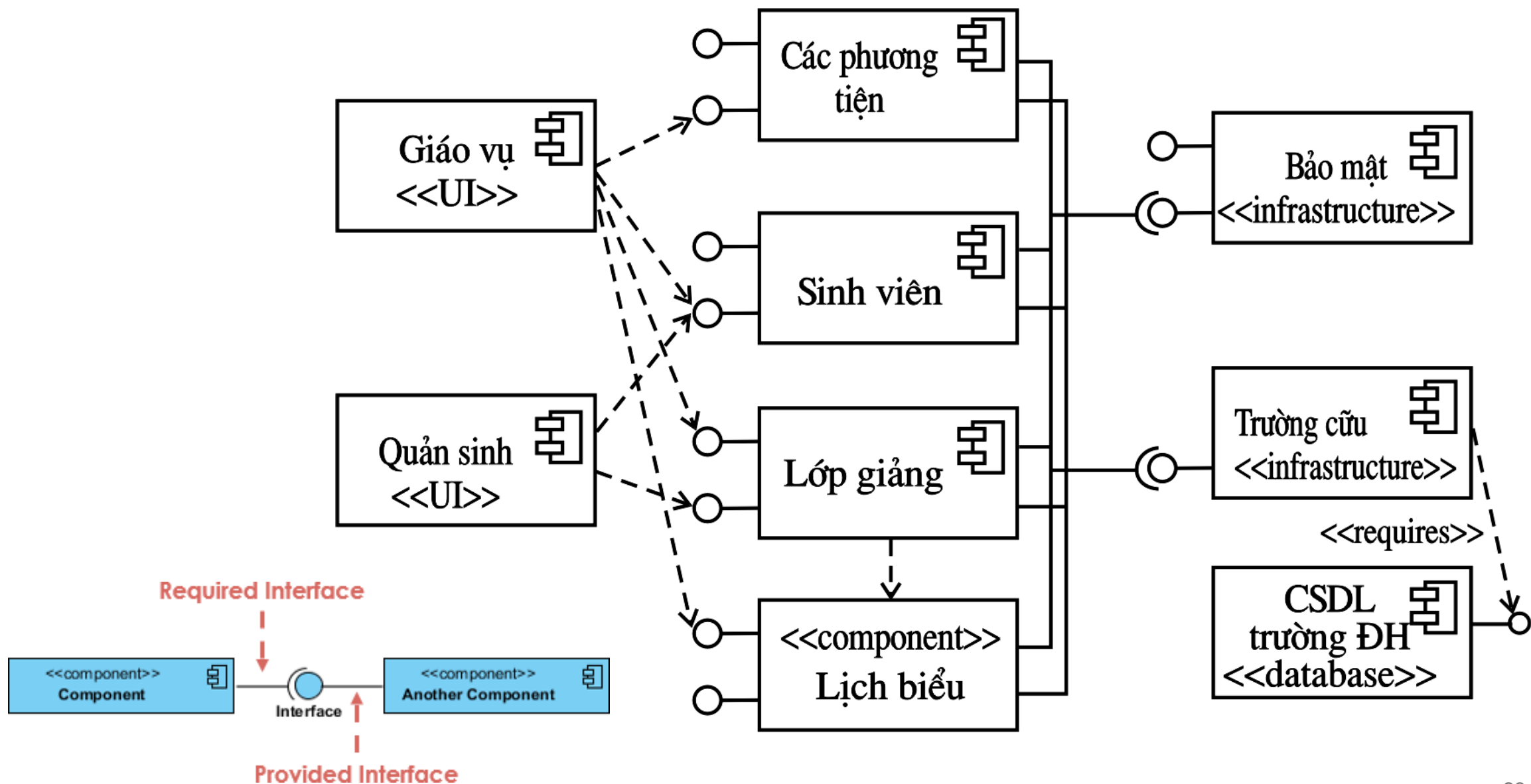
Có 3 loại thành phần:

- **Các thành phần triển khai** (deployment components): Đó là các thành phần cần và đủ để tạo nên một hệ thống khả thi, như là các thư viện động (DLL) và các mã thực thi (executable).
- **Các thành phần sản phẩm làm việc** (work product components): Đó là các thành phần có từ quá trình phát triển hệ thống, bao gồm các tệp mã nguồn, các tệp dữ liệu, từ đó mà ta đã tạo lập ra các thành phần triển khai. Các thành phần này không trực tiếp tham gia vào hệ thống thực thi, nhưng không có chúng thì không tạo được hệ thống thực thi.
- **Các thành phần thực hiện** (execution components): Đó là các thành phần được tạo nên như là một kết quả của một hệ thực hiện (vd: một thành phần .JAR)

Để tổ chức các thành phần lại với nhau, ta có hai cách:

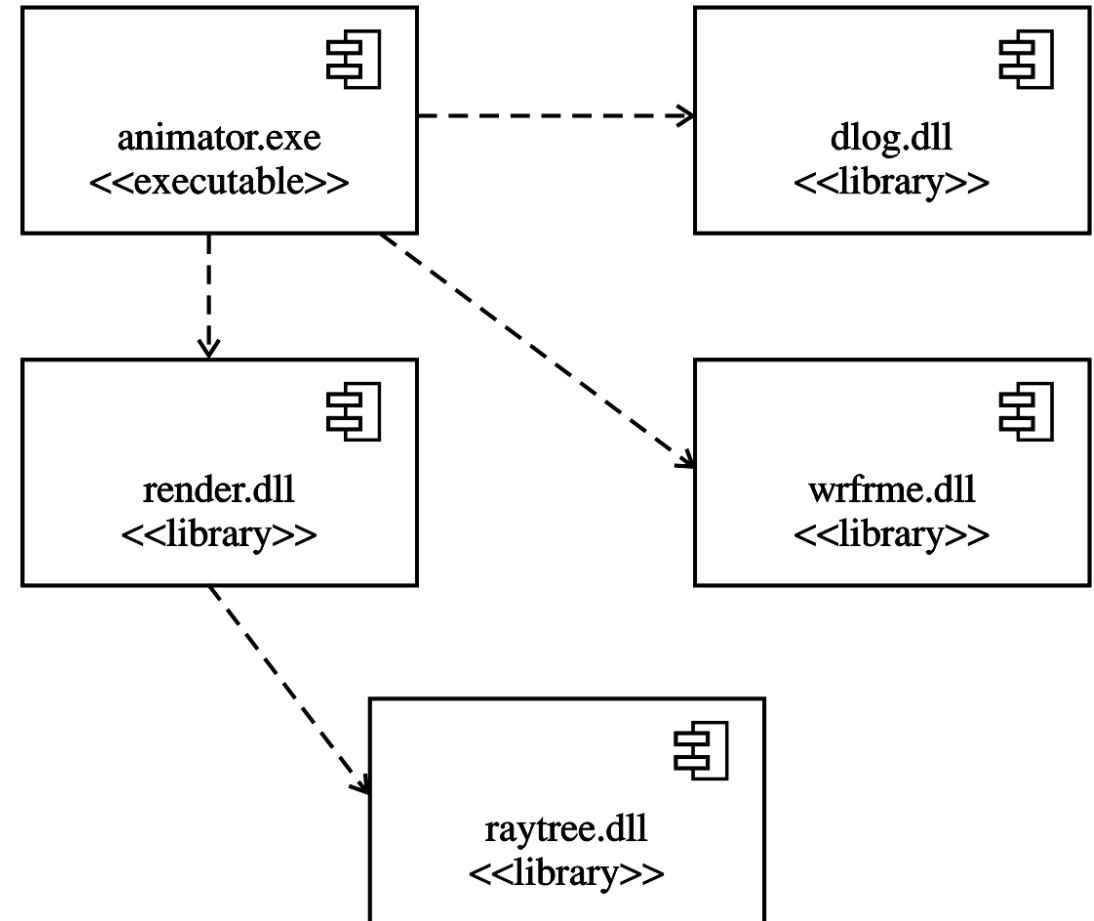
- Gom các thành phần vào các gói, nghĩa là đưa chúng vào các hệ thống con; hoặc
- Thiết lập các mối liên quan phụ thuộc giữa chúng, và như thế ta có một Biểu đồ thành phần.

| Biểu đồ thành phần



| Mô hình hoá các thành phần khả thi và thư viện

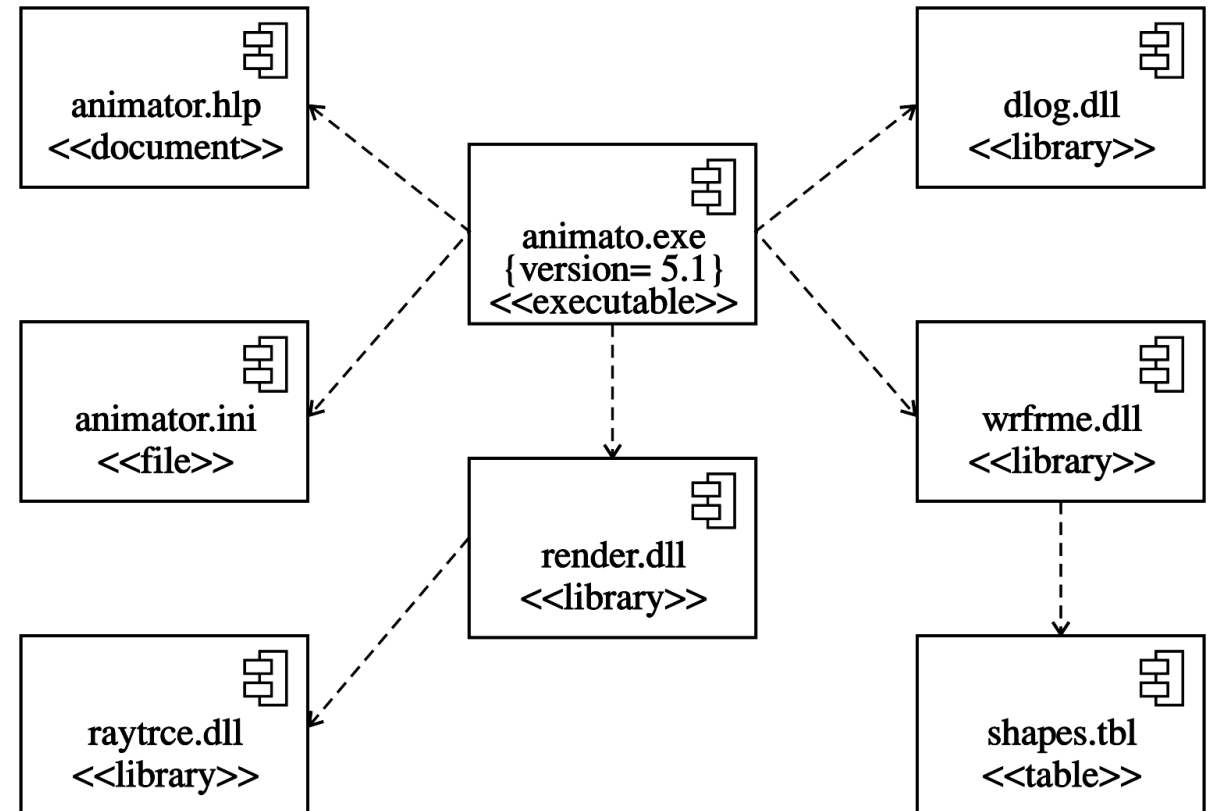
- Mục đích chính của việc sử dụng Biểu đồ thành phần là để mô hình hoá các thành phần triển khai, tạo nên sự cài đặt của hệ thống.
 - Nếu ta muốn cài đặt một hệ thống chỉ gồm đúng một tệp khả thi (.EXE), thì ta chẳng cần dùng tới thành phần
 - Nhưng nếu hệ thống gồm nhiều tệp khả thi và liên kết với các thư viện đối tượng, thì ta cần dùng biểu đồ thành phần để giúp hiển thị, đặc tả, thành lập và tư liệu hoá các quyết định đối với hệ thống vật lý



Một biểu đồ các thành phần triển khai

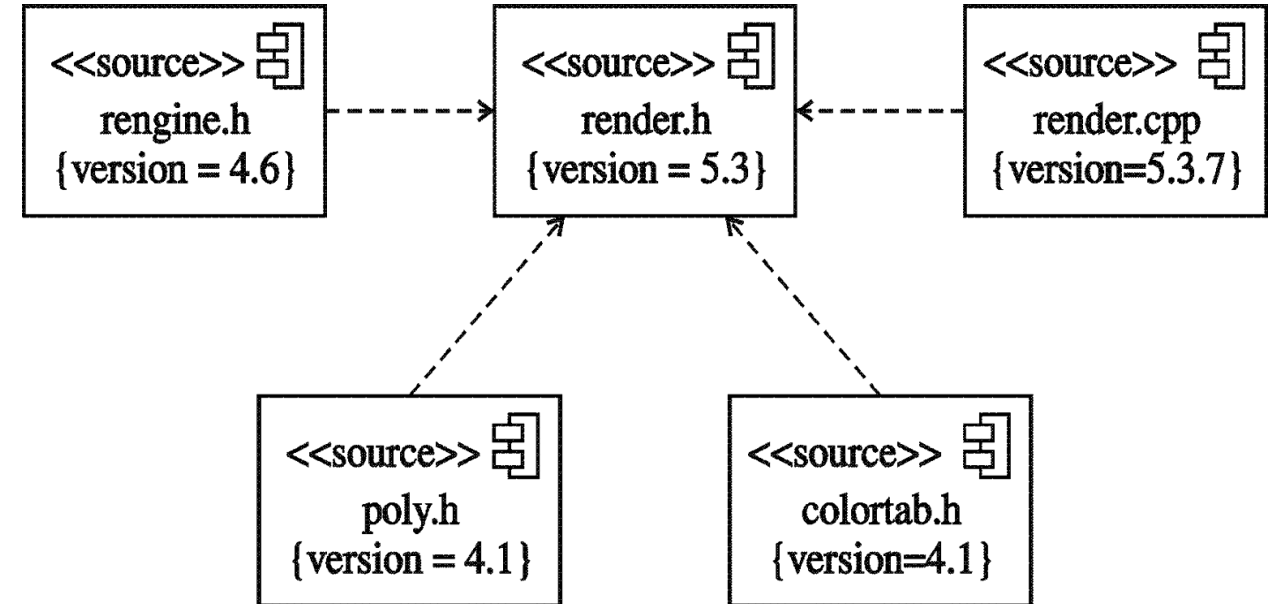
| Mô hình hoá các bảng, các tệp và các tài liệu

- Bên cạnh các tệp khả thi và thư viện tạo nên phần chạy được của hệ thống, thì còn nhiều thành phần bố trí khác, gọi là các thành phần phụ trợ, cần cho việc cài đặt hệ thống
 - Ví dụ: Để cài đặt, ta vẫn cần các tệp dữ liệu, các tài liệu trợ giúp, các scripts, các tệp log, các tệp khởi tạo tham số cấu hình,...
 - Mô hình hoá các thành phần này cũng là một phần quan trọng để diễn tả hình trạng của hệ thống.



| Mô hình hoá mã nguồn

- Mô hình hoá mã nguồn cũng là một mục đích của việc tạo lập biểu đồ thành phần.
 - Các tệp mã nguồn dùng để chứa các chi tiết về các lớp, các giao diện, các hợp tác và các phần tử logic khác
 - Các tệp mã nguồn tạo nên một bước trung gian để tạo lập các thành phần vật lý/thực thi.
 - Các công cụ (vd: chương trình biên dịch) thường đòi hỏi các tệp mã nguồn phải được tổ chức theo một quy cách nhất định nào đó.



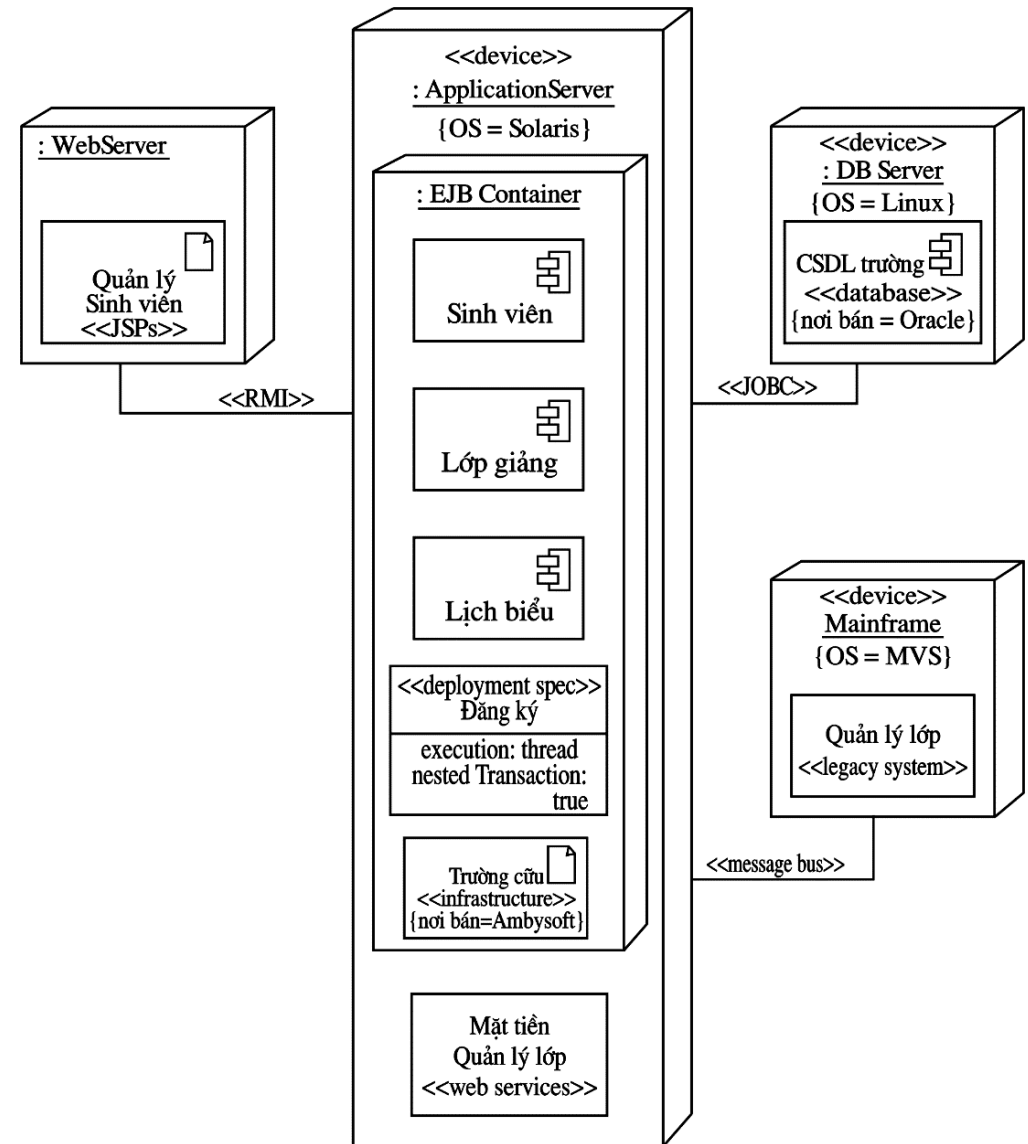
Mô hình hóa mã nguồn xây dựng render.dll

| Bố trí các thành phần khả thi vào các nút phần cứng

- Để bố trí các thành phần phần mềm vào các nút phần cứng, ta dùng Biểu đồ triển khai.
- Một nút (node) là một phần tử vật lý tồn tại vào lúc chạy và biểu diễn cho một tài nguyên tính toán (computational resource). Một nút được biểu diễn bởi một hình hộp, có mang tên. Nếu tên này không gạch dưới, thì nút thể hiện một lớp các tài nguyên. Nếu tên được gạch dưới, thì nút thể hiện một cá thể tài nguyên.
- Có hai mức diễn tả của Biểu đồ triển khai: mức lớp (tương ứng với Biểu đồ lớp) và mức cá thể (tương ứng với Biểu đồ đối tượng).
- Nút có thể là:
 - Một thiết bị (nút phần cứng), thường mang khuôn mẫu `<<device>>` (hoặc cụ thể hơn là `<<processor>>`, `<<console>>`, `<<kiosk>>`, `<<printer>>`,...).
 - Một môi trường thực hiện (nút phần mềm), thường mang khuôn mẫu `<<execution env>>` (vd: `<<EJB Container>>`, `<<J2EEServer>>`).

| Bố trí các thành phần khả thi vào các nút phần cứng

- Nút WebServer chưa có khuôn mẫu, đó là vì người phát triển hệ thống chưa có quyết định – Nút này có thể là một loại phần mềm hoặc là một thiết bị vật lý.
- Các nút có thể chứa các nút khác hoặc các phần mềm.
- Chẳng hạn, nút ApplicationServer chứa nút EJBContainer (một nút phần mềm), và nút này lại chứa ba thành phần phần mềm, một đặc tả bố trí, và một phần mềm.



| Một số khuôn mẫu cho kết nối

- Các kết nối là các mối liên quan giao tiếp giữa các cặp nút, thể hiện về mặt vật lý bằng một đường truyền.
- Mỗi kết nối hỗ trợ cho một hay nhiều giao thức truyền thông, mà ta cần chỉ rõ bằng các khuôn mẫu thích hợp.

Khuôn mẫu	Ý nghĩa
Asynchronous	Một kết nối không đồng bộ
Synchronous	Một kết nối đồng bộ, trong đó bên gửi chờ trả lời từ bên nhận
HTTP	HyperText Transport Protocol, một giao thức Internet
JDBC	Java Database Connectivity, công nghệ Java để kết nối và truy cập CSDL
ODBC	Open Database Connectivity, kết nối và truy cập CSDL
RMI	Remote Method Invocation, một giao thức hỗ trợ lời gọi từ xa
RPC	Remote Procedure Call, lời gọi thủ tục từ xa
Web services	Liên lạc qua các giao thức web services (vd: SOAP)

I 5.6 THIẾT KẾ GIAO DIỆN NGƯỜI DÙNG

- Dựa vào các công cụ tạo lập giao diện người dùng (GUI – Graphical user interface – design tools) ta thành lập sớm và nhanh một nguyên mẫu giao diện người dùng (GUI prototype), có tính thăm dò sự phù hợp, nhằm các mục đích.
 - Tạo ra một môi trường làm việc cụ thể, dễ tiếp xúc, dễ làm thử, làm cho người dùng trở nên yên tâm hơn, và năng động hơn trong việc đóng góp cho việc phát triển hệ thống.
 - Qua quá trình dùng thử, ta thu thập được nhiều ý kiến phản hồi có ích từ phía người dùng.
 - Sớm phát hiện được các yêu cầu hay chức năng bị bỏ sót, sớm nhìn thấy các điểm yếu, chỗ khó khăn nhất của hệ thống.
- Thiết kế giao diện không phải là các ảnh chụp màn hình hệ thống sau cài đặt!

| Thiết kế giao diện người dùng

- Như đã biết, cứ mỗi cặp (tác nhân, ca sử dụng) liên quan, có ít nhất một lớp biên để chuyển đổi các thông tin vào/ra.
 - Thể hiện của lớp biên chính là giao diện mà bây giờ ta cần phải mô tả.
- Muốn mô tả các lớp biên, ta xem lại từng bước trong kịch bản của mỗi ca sử dụng – để xác định các phần tử của giao diện:
 - Xét nội dung của tương tác giữa tác nhân và hệ thống.
 - Các thông tin vào và Các thông tin ra.
 - Các hành động được yêu cầu.

| Mô tả các giao diện của hệ thống

Bản mô tả một giao diện thường chứa các điểm sau:

- Tên của giao diện;
- Mô tả tóm tắt nội dung của giao diện;
- Mức độ phức tạp của giao diện (phức tạp/chuẩn/đơn giản);
- Ghi chú thêm (nếu có);
- Thiết kế chi tiết của giao diện, tùy thuộc vào 4 loại giao diện sau:
 - Các giao diện hội thoại (với người dùng).
 - Các giao diện hiển thị thông tin (kết quả xử lý, báo cáo,...).
 - Các giao diện dữ liệu từ/đến các hệ thống ngoài.
 - Các giao diện chức năng đến các hệ thống ngoài.

| Làm nguyên mẫu giao diện

- Làm nguyên mẫu giao diện nên được bắt đầu càng sớm càng tốt.
 - Có thể bắt đầu làm nguyên mẫu giao diện ngay khi xong phân tích các ca sử dụng.
- Có nhiều bộ tạo lập giao diện người dùng (GUI builders) cho phép làm các nguyên mẫu giao diện mà không tốn mấy công sức.
 - <https://www.figma.com>
 - <https://icons8.com/lunacy>
- Đối với phiên bản đầu của nguyên mẫu giao diện, thì các trường là rỗng hoặc mang giá trị minh họa, các nút và các phần tử tương tác có thể chưa cần có hiệu ứng tương tác rõ ràng.
- Qua các vòng lặp tiếp theo (của việc làm nguyên mẫu), thì giao diện dần trở nên sinh động hơn (dần đi tới phương án cuối).
- Như vậy, người dùng có thể sớm làm việc thử với các nguyên mẫu giao diện.

| Làm nguyên mẫu giao diện

- Nguyên mẫu giao diện chỉ có ý nghĩa thăm dò => nên làm nhanh và không nên cầu toàn.
 - Chưa nên chú ý nhiều về trình bày và mỹ thuật, mà cần chú ý đến nội dung (các trường và các điều khiển của giao diện) và luồng dẫn dắt từ phần tử giao diện này sang phần tử giao diện khác (screens flow).
- Quá trình phát triển nguyên mẫu giao diện nên được làm song song với quá trình phân tích và thiết kế, và nên hỗ trợ cho phân tích và thiết kế.
- Nguyên mẫu giao diện nói chung chỉ là mặt ngoài của hệ thống, không phản ánh hết tầm sâu (các chi tiết hoạt động) của hệ thống
 - Cần nói rõ với người dùng là nguyên mẫu không phải là hệ thống hoàn chỉnh.

| Bài tập cá nhân

1. Dựa vào bài tập Chương 4, hãy trình bày kiến trúc phân cụm và các gói cho hệ thống.
2. Hãy trình bày kiến trúc phân cụm và các gói cho hệ quản lý học theo tín chỉ và chỉ ra cách chọn lựa của mình.
3. Hãy trình bày kiến trúc phân cụm và các gói cho hệ quản lý thư viện và chỉ ra cách chọn lựa của mình.
4. Hãy trình bày kiến trúc phân cụm và các gói cho hệ quản lý đặt thuê xe và chỉ ra cách chọn lựa của mình.
5. Hãy trình bày kiến trúc phân cụm và các gói cho hệ quản lý siêu thị mua bán sách và chỉ ra cách chọn lựa của mình.

